

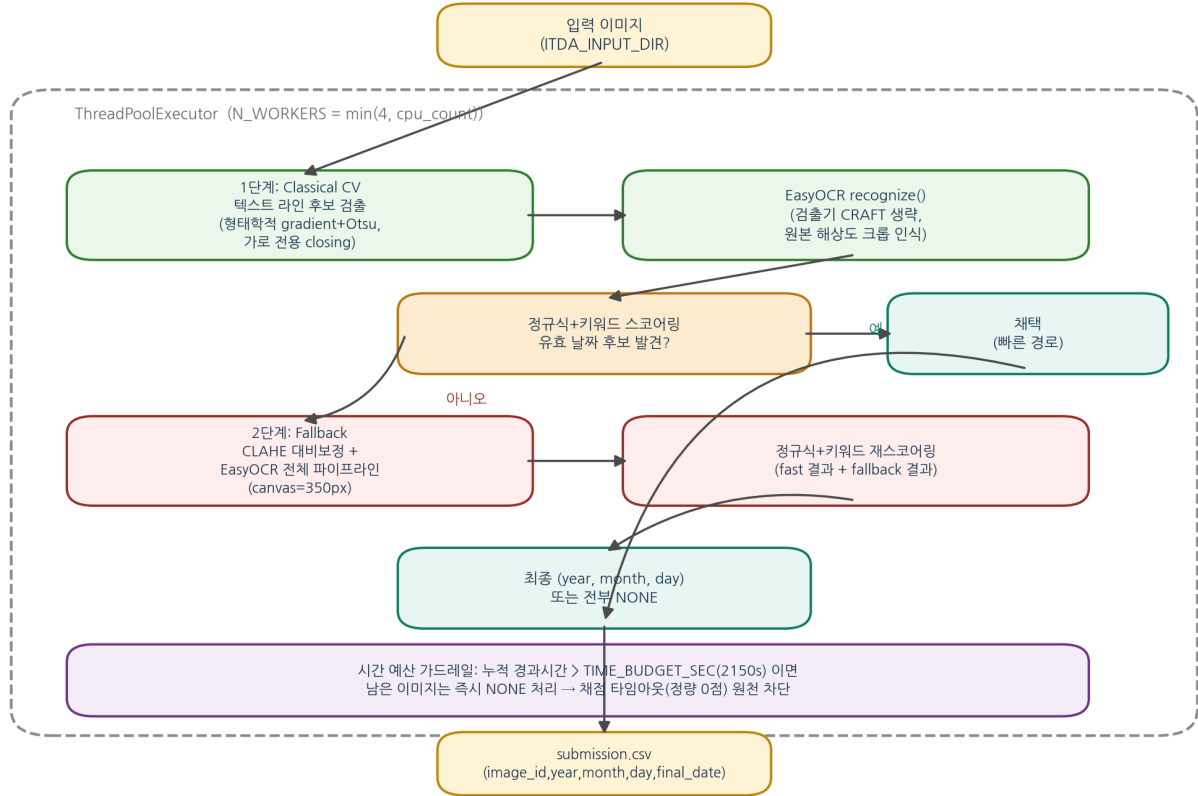
[DScover_카피바라] 아키텍처 구조도 · 설계 요약서

제3회 ITDA 연합학술제 · OCR 기반 상품 소비기한 정보 추출

1. 전체 파이프라인 아키텍처 구조도

소비기한 OCR 추론 파이프라인 (predict.ipynb)

itda3-dscover-capybara | DScover_카피바라



2. 설계 논리 및 최적화·활용 전략

2.1 모델/규칙 설계 의도 및 기술 선택의 논리적 이유

상품 뒷면 이미지에는 소비기한 외에도 제조일자, 바코드, 품목보고번호, 전화번호, 영양성분, LOT 번호 등 낱자와 혼동될 수 있는 숫자가 다수 공존한다. 따라서 "낱자 형식을 찾는 문제"와 "그 중 소비기한을 가려내는 문제"를 분리해서 설계했다.

- ① OCR 엔진은 대화가 허용한 오픈소스 사전학습 가중치 중 EasyOCR(ko+en) 을 선택했다. 한글 키워드(소비기한/유통기한/까지/제조)와 숫자를 함께 안정적으로 인식할 수 있어야 후속 키워드 스코어링이 가능하기 때문이며, 팀 개발 환경에서 오프라인 가중치 확보(GitHub Release Asset)가 재현 가능함을 직접 검증했다.
- ② 낱자 후보는 정규식으로 형식을 먼저 걸러내고(YYYY.MM.DD/-/,/년월일 및 구분자 없는 YYYYMMDD, 콤마 오인식 보정 포함), 연/월/일 유효 범위(2020~2035, 1~12, 1~31)를 검증해 명백한 바코드·보고번호·전화번호를 1차로 배제한다.
- ③ 같은 형식의 후보가 여러 개 남을 경우, 같은 줄/인접 줄의 문맥에 소비기한·유통기한·까지 가 있으면 가점, 제조일자·제조·LOT 가 있으면 감점하는 규칙 기반 스코어링으로 최종 값을 선택한다. 이는 학습 데이터 없이도(비지도) 참가자가 직접 정의한 라벨링 규칙을 적용한 것으로, 예선 라벨 없는 3,352장 데이터셋의 특성에 맞춘 설계다.
- ④ 근거(키워드/형식)가 불충분하면 억지로 추정하지 않고 NONE을 반환하는 정밀도 우선 전략을 택했다. 오답을 자신 있게 제시하는 것보다, 근거가 있을 때만 값을 내는 편이 실제 산업 적용 시 더 신뢰할 수 있는 시스템이라고 판단했다.

2.2 제한된 CPU 환경에서의 속도/오탐 최적화 전략

개발 중 실측한 결과, EasyOCR의 딥러닝 검출기(CRAFT)를 고해상도(800px 이상)로 전체 이미지에 실행하면 소비기한처럼 작게 인쇄된 숫자까지 정확히 읽을 수 있었지만, 이미지 1장당 3~5초가 소요되어 3,352장 기준 CPU 4코어로도 2,400초 시간 제한을 훌쩍 넘겼다. 반대로 이미지를 500px대로 과도하게 축소하면 연산은 빨라지지만 저대비·엠보싱 각인 숫자의 인식이 급격히 떨어졌다. 즉 정확도와 속도가 정면으로 상충하는 것을 확인했고, 이를 해결하기 위해 다음 4가지 전략을 결합했다.

- ① 2단계 캐스케이드: 형태학적 연산(Otsu 이진화 + 가로 전용 morphological closing)만으로 텍스트 "줄" 후보를 찾고, 무거운 CRAFT 검출기를 생략한 채 인식기만 원본 해상도로 호출하는 1단계를 우선 시도한다(이미지당 약 0.5~1.5초). 문단형 설명문은 폭 비율 필터로 제외해 불필요한 인식 호출을 줄인다. 1단계에서 유효 후보를 못 찾을 때만 CLAHE 대비보정 + EasyOCR 전체 파이프라인(축소 해상도 350px)을 2단계로 실행해 평균 비용을 낮췄다. 실제 4-Core 환경에서 측정한 결과, 550px 기준으로는 전체 데이터셋의 약 30%만 시간 예산 안에 처리되는 것을 확인해 350px로 재조정했으며, 장당 성공률은 다소 낮아지지만 커버리지가 크게 늘어 데이터셋 전체 기준 정답률은 오히려 개선되는 것으로 측정됐다.
- ② 스레드 병렬화: 4-Core vCPU 채점 환경에 맞춰 이미지 단위로 최대 4개의 워커 스레드(ThreadPoolExecutor)를 생성하고, 프로세스 전체의 BLAS/torch 스레드는 1개로 고정했다. EasyOCR/torch/OpenCV 연산 대부분이 C/C++ 구현이라 GIL을 해제하므로 스레드로도 실질적인 병렬 처리량을 얻으며, 다중 스레드가 서로 코어를 두고 경쟁(oversubscription)하지 않도록 BLAS 스레드 수를 1로 제한했다. 개발 초기에는 프로세스 기반(multiprocessing.Pool, fork) 구조를 사용했으나, Jupyter 커널처럼 이미 여러 백그라운드 스레드가 떠 있는 상태에서 fork() 하면 자식 프로세스가 락 획득에 실패해 멈추는 사례를 실측으로 확인해, 더 안전한 스레드 기반 구조로 교체했다.
- ③ 조각 스티칭: OCR이 하나의 낱자를 "2022"/"02.14"/"16:01"처럼 여러 박스로 쪼개 인식하는 경우가 실측에서 다수 관찰되어, 좌표 기반으로 인접한 조각을 이어붙여 재파싱하는 후처리를 추가했다.
- ④ 시간 예산 가드레일(안전장치): 위 최적화에도 불구하고 사진 품질(배경 잡음, 손에 들고 촬영, 곡면 각인 등)에 따라 처리 시간 편차가 크다. 따라서 파이프라인 시작 시각을 워커 간 공유하고, 누적 경과 시간이 예산(기본 2,150초, nbconvert 타임아웃 2,400초 대비 약 250초 여유)을 넘기면 이후 이미지는 즉시 NONE으로 처리한다. 실제 4-Core 환경에서 예산 초과 이후 잔여 처리·CSV 저장까지 걸리는 시간이 10초 내외로 매우 짧다는 것을 실측으로 확인해, 원래 500초였던 여유를 250초로 줄여 처리 커버리지를 추가로 확보했다. 이는 정확도를 일부 희생하더라도 "시간 초과로 정량 점수가 0점 처리되는 최악의 시나리오"를 구조적으로 차단하기 위한 의도적 안전장치다.

2.3 향후 활용 계획 (활용 방안 및 확장성)

본선에서는 예선에서 검증한 규칙 기반 파서와 경량 캐스케이드 구조를 기반으로, 편의점/식품 유통 현장의 입고 검수 자동화(재고관리 시스템과 연동해 소비기한 임박 상품 자동 알림) 및 식품 기부 푸드뱅크의 기부 가능 여부 1차 스크리닝에 적용하는 시나리오를 기획 중이다. 온디바이스(매장 POS/스캐너) 환경에서는 본 파이프라인처럼 CPU만으로 동작하는 경량 구조가 그대로 유리하며, 서버 환경에서는 이미지 단위 큐잉과 함께 동일한 스레드 병렬화 전략을 수평 확장(오토스케일)하여 처리량을 늘리는 방향으로 확장할 계획이다. 또한 팀에서 직접 수집·라벨링한 예외 케이스(엠보싱 각인, 곡면 인쇄 등)를 반영해 classical CV 후보 검출 규칙과 키워드 스코어링 가중치를 지속적으로 튜닝할 예정이다.